

Review this cheat sheet to learn more about the various data preprocessing tasks in Pandas, their descriptions, syntax, and examples.

Cheat Sheet : Data Preprocessing Tasks in Pandas

Load CSV data

Read data from a CSV file into a Pandas DataFrame

Syntax:

```
pd.read_csv('filename.csv')
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
df_can=pd.read_csv('data.csv')
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Handling Missing Values

Drop rows with missing values

Syntax:

```
df.dropna()
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
df_can.dropna()
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Handling Missing Values

Fill missing values with a specified value

Syntax:

```
df.fillna(value)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
df_can.fillna(0)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Removing Duplicates

Remove duplicate rows

Syntax:

```
df.drop_duplicates()
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
df_can.drop_duplicates()
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Renaming Columns

Rename one or more columns

Syntax:

```
df.rename(columns={'old_name': 'new_name'})
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
df_can.rename(columns={'Age': 'Years'})
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Selecting Columns

Select a single column

Syntax:

```
df['column_name'] or df.column_name
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
df_can.Age or df_can['Age']
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Selecting Columns

Select multiple columns

Syntax:

```
df[['col1', 'col2']]
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
df_can[['Name', 'Age']]
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Filtering Rows

Filter rows based on a condition

Syntax:

```
df[df['column'] > value]
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
df_can[df_can['Age'] > 30]
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Applying Functions to Columns

Apply a function to transform values in a column

Syntax:

```
df['column'].apply(function_name)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
df_can['Age'].apply(lambda x: x + 1)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Creating New Columns

Create a new column with values derived from existing ones

Syntax:

```
df['new_column'] = expression
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
df_can['Total'] = df_can['Quantity'] * df_can['Price']
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Grouping and Aggregating

Group rows by a column and apply aggregate functions

Syntax:

```
df.groupby('column').agg({'col1': 'sum', 'col2': 'mean'})
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
df_can.groupby('Category').agg({'Total': 'mean'})
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Sorting Rows

Sort rows based on a column

Syntax:

```
df.sort_values('column', ascending=True/False)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
df_can.sort_values('Date', ascending=True)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Displaying First n Rows

Show the first n rows of the DataFrame

Syntax:

```
df.head(n)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
df_can.head(3)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Displaying Last n Rows

Show the last n rows of the DataFrame

Syntax:

```
df.tail(n)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
df_can.tail(3)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Checking for Null Values

Check for null values in the DataFrame

Syntax:

```
df.isnull()
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
df_can.isnull()
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Selecting Rows by Index

Select rows based on integer index

Syntax:

```
df.iloc[index]
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
df_can.iloc[3]
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Selecting Rows by Index

Select rows in a specified range

Syntax:

```
df.iloc[start:end]
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
df_can.iloc[2:5]
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Selecting Rows by Label

Select rows based on label/index name

Syntax:

```
df.loc[label]
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
df_can.loc['Label']
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Selecting Rows by Label

Select rows in a specified label/index range

Syntax:

```
df.loc[start:end]
```




Information:
Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
df_can.loc['Age':'Quantity']
```



Information:
Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Summary Statistics

Generates descriptive statistics for numerical columns

Syntax:

```
df.describe()
```



Information:
Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Example:

```
df_can.describe()
```



Information:
Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Review this cheat sheet to learn more about the various plot libraries, their main purpose, key features, programming languages, level of customization, dashboard capabilities, and the types of plots possible in them.

Cheat Sheet : Plot Libraries

Library	Main Purpose	Key Features	Programming Language	Level of Customization	Dashboard Capabilities	Types of Plots Possible
Matplotlib	General-purpose plotting	Comprehensive plot types and variety of	Python	High	Requires additional components and customization	Line plots, scatter plots, bar charts,

Library	Main Purpose	Key Features	Programming Language	Level of Customization	Dashboard Capabilities	Types of Plots Possible
		customization options				histograms, pie charts, box plots, heatmaps, etc.
Pandas	Fundamentally used for data manipulation but also has plotting functionality	Easy to plot directly on Panda data structures	Python	Medium	Can be combined with web frameworks for creating dashboards	Line plots, scatter plots, bar charts, histograms, pie charts, box plots, etc.
Seaborn	Statistical data visualization	Stylish, specialized statistical plot types	Python	Medium	Can be combined with other libraries to display plots on dashboards	Heatmaps, violin plots, scatter plots, bar plots, count plots, etc.
Plotly	Interactive data visualization	interactive web-based visualizations	Python, R, JavaScript	High	Dash framework is dedicated for building interactive dashboards	Line plots, scatter plots, bar charts, pie charts, 3D plots, choropleth maps, etc.
Folium	Geospatial data visualization	Interactive, customizable maps	Python	Medium	For incorporating maps into dashboards, it can be integrated with other frameworks/libraries	Choropleth maps, point maps, heatmaps, etc.
PyWaffle	Plotting Waffle charts	Waffle charts	Python	Low	Can be combined with other libraries to display waffle chart on dashboards	Waffle charts, square pie charts, donut charts, etc.